

Alunos: José Mário de Melo Brandão Guimarães, Davi Maia, Nikolas Messias, Diogo Florentino, Christopher Lindoso

1. Quais camadas serão testadas?

Serão testadas as camadas unitária, integração, API e UI/E2E, além dos testes manuais exploratórios. Os testes unitários ficam focados nas regras de negócio, enquanto API e integração verificam a comunicação entre os módulos, principalmente em carrinho, estoque, checkout e pagamento.

2. Quais testes serão manuais?

Ficarão manuais principalmente os testes exploratórios, a avaliação visual do checkout em mobile, a validação das mensagens apresentadas ao usuário e cenários com comportamentos externos ou difíceis de manter automatizados. Esses testes ajudam a encontrar problemas que não são facilmente previstos por regras automatizadas.

3. Quais testes serão automatizados?

Serão automatizados os testes unitários das regras de negócio, testes de integração entre módulos, testes de API e os principais fluxos de compra. Entre eles estão cupom, estoque, frete, pagamento e criação do pedido, por serem partes críticas e recorrentes do sistema.

4. Quais ferramentas serão usadas?

JUnit ou Pytest podem ser usados nos testes de código, Postman/Newman para os testes de API e Cypress para os testes de UI/E2E. A execução pode ser integrada ao CI/CD para gerar resultados e evidências das execuções.

5. Como a suíte será organizada?

A suíte será separada por tipo de teste, como API, UI/E2E, dados de teste e relatórios. Dentro de cada grupo, os testes podem ser organizados por funcionalidade, como login, produtos, carrinho, checkout e pedidos, facilitando a manutenção e evitando repetição.

6. Quais testes entram no pipeline?

Os testes unitários devem rodar primeiro, no commit e nas alterações do projeto. No Pull Request entram os testes de API e integração e, em homologação ou antes da release, os E2E dos fluxos mais críticos, como compra, pagamento e criação do pedido.

7. Quais dados serão necessários?

Serão necessários usuários válidos e bloqueados, produtos com e sem estoque, cupons válidos e expirados, dados de pagamento aprovado e recusado e pedidos em diferentes situações. A massa deve ser controlada para evitar falhas falsas e permitir repetir os mesmos cenários.

8. Quais riscos a arquitetura precisa reduzir?

A arquitetura deve reduzir erros de estoque, preço, cupom, pagamento, frete e atualização de pedidos, além de falhas de integração entre os módulos. Também deve diminuir regressões, testes instáveis, falta de evidências e a quantidade de bugs que chegam à produção.

9. Como evitar uma pirâmide de testes invertida?

A maior parte da suíte deve ser formada por testes unitários, por serem mais rápidos e baratos de executar. Depois ficam os testes de integração e API em quantidade moderada,

enquanto os testes de UI/E2E devem ser poucos e focados nos fluxos realmente críticos; os testes manuais ficam para exploração e validações que exigem avaliação humana.